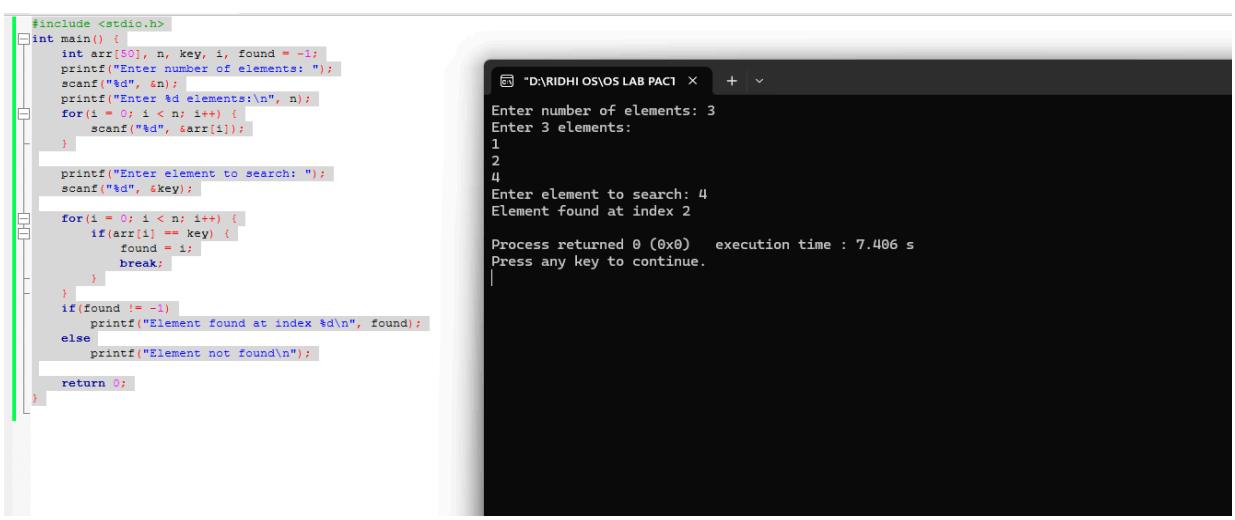
Operating systems(OS lab)

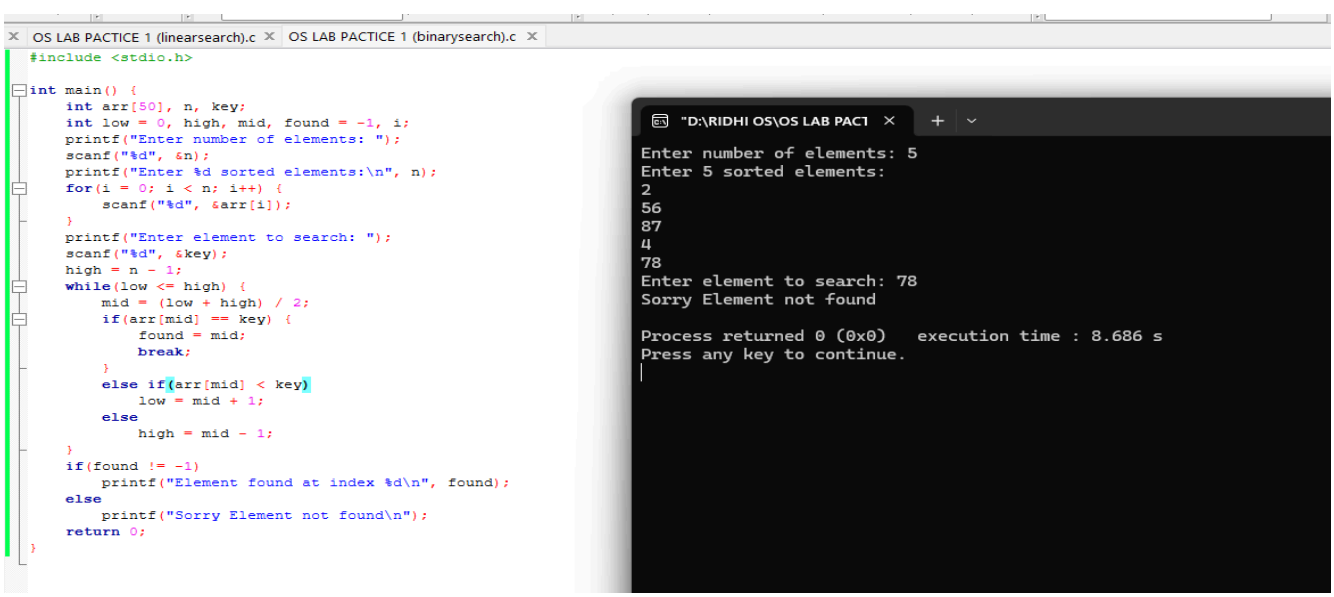
1.linear search

```
#include <stdio.h>
int main() {
    int arr[50], n, key, i, found = -1;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    for(i = 0; i < n; i++) {
        if(arr[i] == key) {
            found = i;
            break;
        }
    }
    if(found != -1)
        printf("Element found at index %d\n", found);
    else
        printf("Element not found\n");

    return 0;
}
```



2.BINARY SEARCH

```c
#include <stdio.h>

int main() {
    int arr[50], n, key;
    int low = 0, high, mid, found = -1, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d sorted elements:\n", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Enter element to search: ");
    scanf("%d", &key);
    high = n - 1;
    while(low <= high) {
        mid = (low + high) / 2;
        if(arr[mid] == key) {
            found = mid;
            break;
        }
        else if(arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    if(found != -1)
        printf("Element found at index %d\n", found);
    else
        printf("Sorry Element not found\n");
    return 0;
}
```



1.FCFS(LAB1)

```
#include <stdio.h>

int main() {
    int n, i, j;
    int at[20], bt[20], ct[20], tat[20], wt[20], rt[20], start[20];
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;

    printf("Enter number of processes (max 20): ");
    scanf("%d", &n);

    if(n > 20 || n <= 0) {
        printf("Invalid number of processes!\n");
        return 1;
    }

    // Input
    for(i = 0; i < n; i++) {
        printf("\nProcess P%d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
    }

    // 🔹 Sort processes by Arrival Time (FCFS rule)
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(at[i] > at[j]) {
                // Swap Arrival Time
                int temp = at[i];
                at[i] = at[j];
                at[j] = temp;

                // Swap Burst Time
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // First process
    start[0] = at[0];
    ct[0] = start[0] + bt[0];

    // Remaining processes
    for(i = 1; i < n; i++) {
        if(ct[i-1] < at[i])
            start[i] = at[i];
```

```
        else
            start[i] = ct[i-1];

        ct[i] = start[i] + bt[i];
    }

    // Calculate TAT, WT, RT
    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        rt[i] = start[i] - at[i];

        avg_wt += wt[i];
        avg_tat += tat[i];
        avg_rt += rt[i];
    }

    // Output
    printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for(i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);

    printf("\nAverage WT = %.2f", avg_wt/n);
    printf("\nAverage TAT = %.2f", avg_tat/n);
    printf("\nAverage RT = %.2f\n", avg_rt/n);

    return 0;
}
```

```
, start[20];
"D:\RIDHI OS\FCFS(LAB1).e

Enter number of processes (max 20): 3

Process P1
Arrival Time: 2
Burst Time: 4

Process P2
Arrival Time: 3
Burst Time: 5

Process P3
Arrival Time: 6
Burst Time: 4

P       AT      BT      CT      TAT     WT      RT
P1      2       4       6       4       0       0
P2      3       5       11      8       3       3
P3      6       4       15      9       5       5

Average WT = 2.67
Average TAT = 7.00
Average RT = 2.67

Process returned 0 (0x0)   execution time : 22.443 s
Press any key to continue.
```

2.SFJ(NON PREMTIVE)(LAB1)

```
#include <stdio.h>

int main() {
    int n, time = 0, completed = 0;
    int at[20], bt[20], ct[20], tat[20], wt[20], rt[20];
    int visited[20] = {0};
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        printf("\nProcess P%d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
    }

    while(completed < n) {
        int idx = -1;
        int min_bt = 9999;

        for(int i = 0; i < n; i++) {
            if(at[i] <= time && visited[i] == 0 && bt[i] < min_bt) {
                min_bt = bt[i];
                idx = i;
            }
        }

        if(idx == -1) {
            time++;
        } else {
            rt[idx] = time - at[idx];
            time += bt[idx];
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            avg_wt += wt[idx];
            avg_tat += tat[idx];
            avg_rt += rt[idx];

            visited[idx] = 1;
            completed++;
        }
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for(int i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
```

```
    printf("\nAverage WT = %.2f", avg_wt/n);
    printf("\nAverage TAT = %.2f", avg_tat/n);
    printf("\nAverage RT = %.2f\n", avg_rt/n);

    return 0;
}
```

```
Process P1
Arrival Time: 2
Burst Time: 5

Process P2
Arrival Time: 3
Burst Time: 6

Process P3
Arrival Time: 1
Burst Time: 7

Process P4
Arrival Time: 3
Burst Time: 8

P       AT      BT      CT      TAT     WT      RT
P1      2       5       13      11      6       6
P2      3       6       19      16      10      10
P3      1       7       8       7       0       0
P4      3       8       27      24      16      16

Average WT = 8.00
Average TAT = 14.50
Average RT = 8.00

Process returned 0 (0x0)   execution time : 15.571 s
Press any key to continue.
```

3.SJF(PREMTIVE)(LAB1)

```
#include <stdio.h>

int main() {
    int n, time = 0, completed = 0;
    int at[20], bt[20], rt[20], ct[20], tat[20], wt[20], response[20];
    int started[20] = {0};
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        printf("\nProcess P%d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        rt[i] = bt[i];   // Remaining time
    }
```

```
    while(completed < n) {
        int idx = -1;
        int min_rt = 9999;   // Large value instead of INT_MAX

        for(int i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0 && rt[i] < min_rt) {
                min_rt = rt[i];
                idx = i;
            }
        }

        if(idx == -1) {
            time++;   // CPU idle
            continue;
        }

        if(started[idx] == 0) {
            response[idx] = time - at[idx];
            started[idx] = 1;
        }

        rt[idx]--;
        time++;

        if(rt[idx] == 0) {
            completed++;
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            avg_wt += wt[idx];
            avg_tat += tat[idx];
            avg_rt += response[idx];
        }
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for(int i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               i+1, at[i], bt[i], ct[i], tat[i], wt[i], response[i]);

    printf("\nAverage WT = %.2f", avg_wt/n);
    printf("\nAverage TAT = %.2f", avg_tat/n);
    printf("\nAverage RT = %.2f\n", avg_rt/n);

    return 0;
}
```

"D:\RIDHI OS\SJF PREMTIV

```
Enter number of processes: 3

Process P1
Arrival Time: 6
Burst Time: 7

Process P2
Arrival Time: 8
Burst Time: 65

Process P3
Arrival Time: 6
Burst Time: 54

P       AT      BT      CT      TAT     WT      RT
P1      6       7       13      7       0       0
P2      8       65      132     124     59      59
P3      6       54      67      61      7       7

Average WT = 22.00
Average TAT = 64.00
Average RT = 22.00

Process returned 0 (0x0)   execution time : 7.459 s
Press any key to continue.
```